ISO/IEC JTC1 SC22 WG21 P0666R0
Date: 2017-06-19
Reply-to: Alasdair Mackintosh alasdair@google.com
Audience: WG21

C++ Latches and Barriers

Contents

1 General [general]

1.1 History [general.history]

This paper has been taken from Section 3 of P0159R0, which describes proposed extensions for concurrency. We propose the section numbered 33.7 below for inclusion in the C++ standard, after 33.6 [futures]. As P0159R0 has been available since 2015 without any recorded objections, and since it represents commonly used concurrency features, we believe it is suitable for inclusion as-is.

A previous version of this paper was published as N4392

1.2 Changes [general.changes]

The following changes have been made to P0159R0.

The main section has been renumbered to reflect its proposed position in the standard.

All functions and classes have been moved into the `std` namespace.

1.3 Additional Proposed Changes [general.proposals]

We propose the following additional wording beyond P0159R0 that clarifies some ambiguities in the original wording. This has been added in the paragraph below so that the committee has the option of accepting P0159R0 as written without any additions, or of including this change.

The wording for `arrive_and_drop` in Section 33.7.6 is ambiguous, as it implies that the first action is to remove the thread from the set of participating threads. Because the completion phase is defined to run in one of the participating threads, this wording would imply that the phase cannot run in any thread that calls `arrive_and_drop`.

We propose the following wording to replace paragraph 13 in Section 33.7.6.

Effects: Removes the current thread from the set of participating threads. If this causes the set to become empty, the barrier type's completion phase is executed. It is unspecified whether the function blocks until the completion phase is ended. [*Note:* If no other thread is blocked in this function, the completion phase executes in the calling thread. — *end note*]

1.4 Main Proposal [general.main]

We propose to add the following clause from the Concurrency TS to the C++20 working draft, as described below.

33.7 Latches and Barriers [coordination]

33.7.1 General [coordination.general]

This section describes various concepts related to thread coordination, and defines the `latch`, `barrier` and `flex_barrier` classes.

33.7.2 Terminology [coordination.terminology]

In this subclause, a *synchronization point* represents a point at which a thread may block until a given condition has been reached.

33.7.3 Latches [coordination.latch]

Latches are a thread coordination mechanism that allow one or more threads to block until an operation is completed. An individual latch is a single-use object; once the operation has been completed, the latch cannot be reused.

33.7.4 Header <latch> synopsis [coordination.latch.synopsis]

```
namespace std {
  class latch {
   public:
    explicit latch(ptrdiff_t count);
    latch(const latch&) = delete;

    latch& operator=(const latch&) = delete;
    ~latch();


    void count_down_and_wait();
    void count_down(ptrdiff_t n = 1);

    bool is_ready() const noexcept;
    void wait() const;

   private:
    ptrdiff_t counter_; // exposition only
  };
} // namespace std
```

33.7.5 Class `latch` [coordination.latch.class]

A latch maintains an internal `counter_` that is initialized when the latch is created. Threads may block at a synchronization point waiting for `counter_` to be decremented to `0`. When `counter_` reaches `0`, all such blocked threads are released.

Calls to `count_down_and_wait()`, `count_down()`, `wait()`, and `is_ready()` behave as atomic operations.

```
explicit latch(ptrdiff_t count);
```

4 *Requires:* `count >= 0`.

5 *Synchronization:* None.

6 *Postconditions:* `counter_ == count`.

```
~latch();
```

8 *Requires:* No threads are blocked at the synchronization point.

9 *Remarks:* May be called even if some threads have not yet returned from `wait()` or `count_down_and_wait()` provided that `counter_` is `0`. [*Note:* The destructor might not return until all threads have exited `wait()` or `count_down_and_wait()`. — *end note*]

```
void count_down_and_wait();
```

11 *Requires:* `counter_ > 0`.

12 *Effects:* Decrements `counter_` by `1`. Blocks at the synchronization point until `counter_` reaches `0`.

13 *Synchronization:* Synchronizes with all calls that block on this latch and with all `is_ready` calls on this latch that return true.

14 *Throws:* Nothing.

```
void count_down(ptrdiff_t n = 1);
```

16 *Requires:* `counter_ >= n` and `n >= 0`.

17 *Effects:* Decrements `counter_` by `n`. Does not block.

18 *Synchronization:* Synchronizes with all calls that block on this latch and with all `is_ready` calls on this latch that return true.

19 *Throws:* Nothing.

```
void wait() const;
```

21 *Effects:* If `counter_` is `0`, returns immediately. Otherwise, blocks the calling thread at the synchronization point until `counter_` reaches `0`.

22 *Throws:* Nothing.

```
is_ready() const noexcept;
```

24 *Returns:* `counter_ == 0`. Does not block.

33.7.6 Barrier types [coordination.barrier]

Barriers are a thread coordination mechanism that allow a *set of participating threads* to block until an operation is completed. Unlike a latch, a barrier is reusable: once the participating threads are released from a barrier's synchronization point, they can re-use the same barrier. It is thus useful for managing repeated tasks, or phases of a larger task, that are handled by multiple threads.

The *barrier types* are the standard library types `barrier` and `flex_barrier`. They shall meet the requirements set out in this subclause. In this description, `b` denotes an object of a barrier type.

Each barrier type defines a *completion phase* as a (possibly empty) set of effects. When the member functions defined in this subclause *arrive at the barrier's synchronization point*, they have the following effects:

1. When all threads in the barrier's set of participating threads are blocked at its synchronization point, one participating thread is unblocked and executes the barrier type's completion phase.
2. When the completion phase is completed, all other participating threads are unblocked. The end of the completion phase synchronizes with the returns from all calls unblocked by its completion.

The expression `b.arrive_and_wait()` shall be well-formed and have the following semantics:

`void arrive_and_wait();`

6 *Requires:* The current thread is a member of the set of participating threads.

7 *Effects:* Blocks and arrives at the barrier's synchronization point. [*Note:* It is safe for a thread to call `arrive_and_wait()` or `arrive_and_drop()` again immediately. It is not necessary to ensure that all blocked threads have exited `arrive_and_wait()` before one thread calls it again. — *end note*]

8 *Synchronization:* The call to `arrive_and_wait()` synchronizes with the start of the completion phase.

9 *Throws:* Nothing.

The expression `b.arrive_and_drop()` shall be well-formed and have the following semantics:

`void arrive_and_drop();`

12 *Requires:* The current thread is a member of the set of participating threads.

13 *Effects:* Removes the current thread from the set of participating threads. Arrives at the barrier's synchronization point. It is unspecified whether the function blocks until the completion phase has ended. [*Note:* If the function blocks, the calling thread may be chosen to execute the completion phase. — *end note*]

14 *Synchronization:* The call to `arrive_and_drop()` synchronizes with the start of the completion phase.

15 *Throws:* Nothing.

16 *Notes:* If all participating threads call `arrive_and_drop()`, any further operations on the barrier are undefined, apart from calling the destructor. If a thread that has called `arrive_and_drop()` calls another method on the same barrier, other than the destructor, the results are undefined.

Calls to `arrive_and_wait()` and `arrive_and_drop()` never introduce data races with themselves or each other.

33.7.7 Header <barrier> synopsis [coordination.barrier.synopsis]

```
namespace std {
  class barrier;
  class flex_barrier;
} // namespace std
```

33.7.8 Class `barrier` [coordination.barrier.class]

`barrier` is a barrier type whose completion phase has no effects. Its constructor takes a parameter representing the initial size of its set of participating threads.

```
class barrier {
 public:
  explicit barrier(ptrdiff_t num_threads);
  barrier(const barrier&) = delete;

  barrier& operator=(const barrier&) = delete;
  ~barrier();

  void arrive_and_wait();
  void arrive_and_drop();
};
```

```
explicit barrier(ptrdiff_t num_threads);
```

[3] *Requires:* `num_threads >= 0.` [*Note:* If `num_threads` is zero, the barrier may only be destroyed. — *end note*]

[4] *Effects:* Initializes the barrier for `num_threads` participating threads. [*Note:* The set of participating threads is the first `num_threads` threads to arrive at the synchronization point. — *end note*]

```
~barrier();
```

[6] *Requires:* No threads are blocked at the synchronization point.

[7] *Effects:* Destroys the barrier.

33.7.9 Class `flex_barrier` [coordination.flexbarrier.class]

`flex_barrier` is a barrier type whose completion phase can be controlled by a function object.

```
class flex_barrier {
 public:
  template <class F>
    flex_barrier(ptrdiff_t num_threads, F completion);
  explicit flex_barrier(ptrdiff_t num_threads);
  flex_barrier(const flex_barrier&) = delete;
  flex_barrier& operator=(const flex_barrier&) = delete;

  ~flex_barrier();

  void arrive_and_wait();
  void arrive_and_drop();

 private:
  function<ptrdiff_t()> completion_;  // exposition only
};
```

The completion phase calls `completion_()`. If this returns `-1`, then the set of participating threads is unchanged. Otherwise, the set of participating threads becomes a new set with a size equal to the returned value. [*Note:* If `completion_()` returns `0` then the set of participating threads becomes empty, and this object may only be destroyed. — *end note*]

```
template <class F>
flex_barrier(ptrdiff_t num_threads, F completion);
```

[4] *Requires:*

—`num_threads >= 0.`
—`F` shall be `CopyConstructible`.
—`completion` shall be Callable (C++14 §[func.wrap.func]) with no arguments and return type `ptrdiff_t`.
—Invoking `completion` shall return a value greater than or equal to `-1` and shall not exit via an exception.

[5] *Effects:* Initializes the `flex_barrier` for `num_threads` participating threads, and initializes `completion_` with `std::move(completion)`. [*Note:* The set of participating threads consists of the first `num_threads` threads to arrive at the synchronization point. — *end note*] [*Note:* If `num_threads` is `0` the set of participating threads is empty, and this object may only be destroyed. — *end note*]

```
explicit flex_barrier(ptrdiff_t num_threads);
```

[7] *Requires:* `num_threads >= 0.`

[8] *Effects:* Has the same effect as creating a `flex_barrier` with `num_threads` and with a callable object whose invocation returns `-1` and has no side effects.

```
~flex_barrier();
```

10 *Requires:* No threads are blocked at the synchronization point.

11 *Effects:* Destroys the barrier.